

RECURSIVIDAD

- Conjuntos definidos recursivamente
 - Fórmulas en lógica de proposiciones
 - Los números naturales
 - Listas
- Correspondencias recursivas
 - Definición de correspondencia y de función recursiva
 - Árbol de dependencia de una correspondencia recursiva
 - Caracterización de funciones recursivas
 - Funciones recursivas definidas sobre números naturales y sobre listas



Bibliografía básica:

- Matemática Discreta. Libro de la asignatura. Cap. 4
- Matemática Discreta y sus aplicaciones.
K.H. Rosen. Mc Graw Hill. Cap. 3
- Matemática discreta y lógica matemática. Hortalá; Leach;
Rodríguez. Ed. Complutense. Cap. 1.3
- Matemática Discreta y Combinatoria. Grimaldi, R.P. Ed. Addison
Wesley. Cap. 4, Cap. 10
- Matemática Discreta Norman L. Biggs. Cap 1.3 y Cap 12

Material de trabajo:

- Matemática Discreta. Problemas. Hoja 3.
- Actividad de Aprendizaje (Moodle)

Este material es un esquema de definiciones y resultados básicos. El objetivo del tema es el análisis (verificación y construcción) de funciones recursivas, por lo que el trabajo está centrado en la resolución de ejercicios y problemas.

MOTIVACIÓN:

Algunas veces se puede obtener la solución de un problema con un conjunto de datos de entrada a partir de la solución del mismo problema pero con un conjunto de datos de cardinal menor que el anterior, o bien del mismo problema con otros datos menores que los iniciales.

Ejemplo :

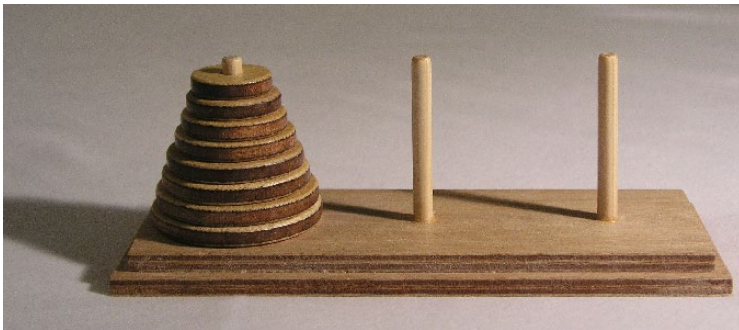
Si $a > b > 0$, $\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$, Algoritmo de Euclides (s.IV a.C.)

Esto permite reducir el problema de trabajar con valores a y b muy grandes, a valores b y $a \bmod b$, que son menores que los iniciales. Si se sigue reduciendo con este criterio, se llega a $\text{mcd}(c, 0) = c$, lo que detiene el proceso de reducción y proporciona el valor buscado.

$$\text{mcd}(33, 21) = \text{mcd}(21, 12) = \text{mcd}(12, 9) = \text{mcd}(9, 3) = \text{mcd}(3, 0) = 3$$

Las torres de Hanoi

Se atribuye a Édouard Lucas (1883), que se inspiró (?) en una leyenda sobre un templo en la India: hay una gran sala con tres postes y 100 discos dorados. Los sacerdotes de Brahma han estado moviendo estos discos de acuerdo con las reglas inmutables de Brahma desde que se construyó. Según la leyenda, cuando se complete el último movimiento del rompecabezas, el mundo se terminará.



$$M(n) = \begin{cases} 1 & \text{si } n = 1 \\ 2M(n-1) + 1 & \text{si } n > 1 \end{cases}$$

$$A(n) = 2^n - 1$$

$$M(n) = A(n) \quad \forall n \geq 1$$

Si la leyenda fuera cierta y los sacerdotes pudieran mover los discos a una velocidad de uno por segundo, hacer esta tarea les llevaría $2^{100} - 1$ segundos, eso es aproximadamente 4.019 cuatrillones de años...

FUNCIONES DEFINIDAS RECURSIVAMENTE

Una **correspondencia recursiva** es una correspondencia que aparece en su propia definición.

Ejemplos:

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \\ n * f(n-1) & \text{si } n \geq 1 \end{cases}$$

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \\ 1 + f((n-1)^2) & \text{si } n \geq 1 \end{cases}$$

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \\ 1 + f(n) & \text{si } n \geq 1 \end{cases}$$

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \\ 1 + f(n-2) & \text{si } n \geq 1 \end{cases}$$

FUNCIONES DEFINIDAS RECURSIVAMENTE

Una **función recursiva** es una correspondencia recursiva que es función y es evaluable a partir de las reglas que la definen en un número finito de pasos.

Dichas reglas se clasifican en:

- Reglas básicas:** definen explícitamente el valor de la función en algunos elementos. Dichos elementos forman el **conjunto de partida o de salida**.
- Reglas recursivas:** definen la función mediante referencias a sí misma.

Ejemplos:

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \text{ R.B.} \\ n * f(n-1) & \text{si } n \geq 1 \text{ R.R.} \end{cases}$$

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \text{ R.B.1} \\ 4 & \text{si } n = 1 \text{ R.B.2} \\ 1 + f(n-2) & \text{si } n \geq 2 \text{ R.R.} \end{cases}$$

FUNCIONES DEFINIDAS RECURSIVAMENTE

Dada una correspondencia recursiva $f : A \rightarrow B$, para cada elemento a de A se denomina **árbol de dependencia de a $Tf(a)$** , a un árbol en cuyos nodos aparecen los elementos en que debe evaluarse f , según la definición recursiva, para obtener el valor $f(a)$.

Ejemplos: Los árboles de dependencia de $n=4$ para las siguientes correspondencias son:

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \quad R.B. \\ n * f(n-1) & \text{si } n \geq 1 \quad R.R. \end{cases}$$

$$f(4) = 4 * f(3) = 4 * 3 * 2 * 1$$

$$f(3) = 3 * f(2) = 3 * 2 * 1$$

$$f(2) = 2 * f(1) = 2 * 1$$

$$f(1) = 1 * f(0) = 1 * 1 = 1$$

$$f(0) = 1$$

$$Tf(4) = \begin{array}{c} 4 \\ | \\ 3 \\ | \\ 2 \\ | \\ 1 \\ | \\ 0 \end{array}$$

FUNCIONES DEFINIDAS RECURSIVAMENTE

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \text{ R.B.1} \\ 4 & \text{si } n = 1 \text{ R.B.2} \\ 1 + f(n-2) & \text{si } n \geq 2 \text{ R.R.} \end{cases}$$

$$\begin{aligned} f(4) &= 1 + f(2) && \text{RR} \\ f(2) &= 1 + f(0) && \text{RR} \\ f(0) &= 1 && \text{RB} \\ f(2) &= 1+1 \\ f(4) &= 1+1+1 = 3 \end{aligned}$$

$$\begin{array}{rcl} \text{Tf}(4) & = & 4 \\ & | & \\ & & 2 \\ & | & \\ & & 0 \end{array}$$

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \\ 1 + f((n-1)^2) & \text{si } n \geq 1 \end{cases}$$

$$\begin{aligned} f(4) &= 1 + f(9) && \text{RR} \\ f(9) &= 1 + f(64) && \text{RR} \\ f(64) &= 1 + f(4096) && \text{RR} \end{aligned}$$

$$\begin{array}{rcl} \text{Tf}(4) & = & 4 \\ & | & \\ & & 9 \\ & | & \\ & & 64 \\ & | & \\ & & 4096 \\ & | & \\ & & \vdots \end{array}$$

Rama infinita

FUNCIONES DEFINIDAS RECURSIVAMENTE

Teorema: Una correspondencia recursiva $f : A \rightarrow B$, es función recursiva si y solo si para todo a de A su árbol de dependencia $Tf(a)$ es finito.

Observación: Si en un árbol de dependencia $Tf(a)$ ocurre alguna de las siguientes condiciones

- Tiene una rama infinita
- Hay un elemento que aparece dos veces en la misma rama
- Llega a un elemento que no está en el conjunto inicial A

El árbol no es finito y por tanto f NO es función recursiva.

Ejemplo: hallar $Tf(1)$ para las correspondencias $f: \mathbb{N} \rightarrow \mathbb{N}$ definidas por:

$$f(n) = \begin{cases} n^2 + 1 & \text{si } n \in \{0, 9, 10\} \\ f(n-1) + f(n-2) & \text{e.o.c.} \end{cases} \quad f(n) = \begin{cases} n + 5 & \text{si } n \in \{0, 9, 10\} \\ f(n-1) + f(n+2) & \text{e.o.c.} \end{cases}$$

FUNCIONES DEFINIDAS RECURSIVAMENTE

Ejemplo: Usar el método de inducción para probar que para la siguiente correspondencia recursiva, el árbol de dependencia $Tf(n)$ es finito para todo $n \in \mathbb{N}$.

$$f: \mathbb{N} \rightarrow \mathbb{N} \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \quad R.B. \\ n * f(n-1) & \text{si } n \geq 1 \quad R.R. \end{cases}$$

Paso base: $f(0) = 1$ (RB) por lo tanto $Tf(0) = 0$ es un árbol de un solo nodo, es finito.

Paso de inducción: Sea $n \geq 0$, tal que $Tf(n)$ es finito (H.I.). Hay que probar que $Tf(n+1)$ también es finito. Por la RR se tiene que $f(n+1) = (n+1) * f(n)$ y por lo tanto el árbol es:

$$Tf(n+1) = \begin{array}{c} n+1 \\ | \\ Tf(n) \end{array}$$

La H.I. asegura que $Tf(n)$ es finito, por lo tanto $Tf(n+1)$ también es un árbol finito.

Queda probado que $Tf(n)$ es finito para todo n

FUNCIONES DEFINIDAS RECURSIVAMENTE

Ejemplo: Usar el método de inducción para probar que la siguiente correspondencia es función.

$$f: N \rightarrow N \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \text{ R.B.1} \\ 4 & \text{si } n = 1 \text{ R.B.2} \\ f(n-1) + f(n-2) & \text{si } n \geq 2 \text{ R.R.} \end{cases}$$

Paso base: $f(0) = 1$ (RB1) por lo tanto $Tf(0) = 0$ es un árbol de un solo nodo, es finito.
 $f(1) = 4$ (RB2) por lo tanto $Tf(1) = 1$ es un árbol de un solo nodo, es finito.

Paso de inducción: Sea $n \geq 0$, tal que $Tf(n)$ y $Tf(n+1)$ son finitos (H.I.). Hay que probar que $Tf(n+2)$ también es finito. Por la RR se tiene que $f(n+2) = f(n+1) + f(n)$ y por lo tanto el árbol es:

$$\begin{array}{c} Tf(n+2) = n+2 \\ \swarrow \quad \searrow \\ Tf(n+1) \quad Tf(n) \end{array}$$

La H.I. asegura que $Tf(n)$ y $Tf(n+1)$ son finitos, por lo tanto $Tf(n+2)$ también es un árbol finito.

Queda probado que $Tf(n)$ es finito para todo n .

FUNCIONES DEFINIDAS RECURSIVAMENTE

Ejemplo: Dar una definición recursiva de $f(n) = 5^n$, $n \in \mathbb{N}$, y probar por inducción que coincide con f .

Ejemplo: Conjeturar una forma explícita para la correspondencia recursiva siguiente. Probar por inducción que esa expresión coincide con la definición recursiva.

$$f: \mathbb{N} \rightarrow \mathbb{N} \quad f(n) = \begin{cases} 1 & \text{si } n = 0 \quad R.B. \\ n * f(n-1) & \text{si } n \geq 1 \quad R.R. \end{cases}$$

CONJUNTOS DEFINIDOS RECURSIVAMENTE

Un conjunto está definido recursivamente si su definición consta de dos tipos de reglas:

- **Reglas básicas:** establecen que ciertos elementos están en el conjunto.
- **Reglas recursivas:** establecen que un elemento está en el conjunto si otros también lo están.

Ejemplos:

- El conjunto de fórmulas proposicionales
- El conjunto de los números naturales N
 - a. Regla básica: $0 \in N$
 - b. Regla recursiva: si $n \in N$ entonces $n+1 \in N$
- El conjunto de los números pares A
 - a. Regla básica: $0 \in A$
 - b. Regla recursiva: si $n \in N$ entonces $n+2 \in A$

CONJUNTOS DEFINIDOS RECURSIVAMENTE

Problemas asociados:

- Construir definiciones recursivas
- Determinar si un elemento pertenece a un conjunto

Ejemplos:

1- Dar una definición recursiva de los múltiplos naturales de 5.

2- Se considera el conjunto A de cadenas de bits definido por las reglas:

- a. RB: La cadena 01 está en A
- b. RR: Si α está en A, entonces $0\alpha 1$ está en A

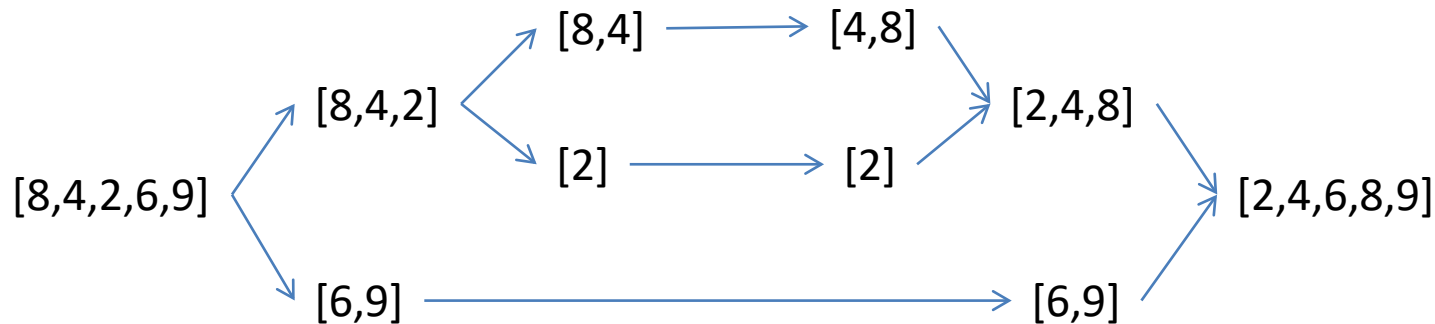
Generar algunos elementos de A y describir el tipo de cadenas que contiene A.

Analizar si 000111 y 00111 son cadenas de A

CONJUNTOS DEFINIDOS RECURSIVAMENTE

Ejemplo:

Si se desea **ordenar** una lista de n números, se puede partir la lista en dos listas de tamaños lo más parecidos posibles, y, una vez ordenadas estas dos, mezclarlas adecuadamente. La estrategia de partir las listas se puede repetir hasta llegar a listas con uno o dos elementos, cuya ordenación es inmediata. El último paso sería ir mezclando listas adecuadamente.



Los ingredientes clave de esta estrategia son:

- **reducir el problema** a otro de la misma naturaleza pero de "tamaño" menor estableciendo la relación entre ambos
- llegar a **casos iniciales** cuya solución es **fácil** de obtener

CONJUNTOS DEFINIDOS RECURSIVAMENTE

El conjunto de las listas

Aproximación intuitiva

$[1, 2, 3, 4]$ $[1, 2, 2, 2]$ $[\]$ $[[2, 3], [2, 6], [3, 5], [4, 7]]$ $[[3, [4, 7]], [5, 6], 2]$

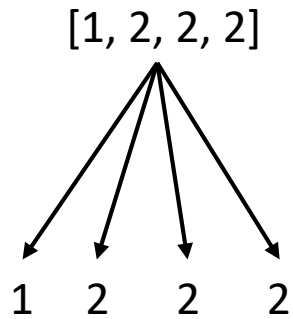
Sea Σ un conjunto no vacío. El conjunto de listas sobre Σ , $LIST(\Sigma)$, se define mediante las siguientes reglas:

Reglas básicas: Si $a_0, a_1, \dots, a_n \in \Sigma$, entonces $[a_0, a_1, \dots, a_n]$ es una lista. La secuencia $[\]$ es una lista, llamada **lista vacía**. Todas estas listas se denominan **listas planas** $LIST_p(\Sigma)$

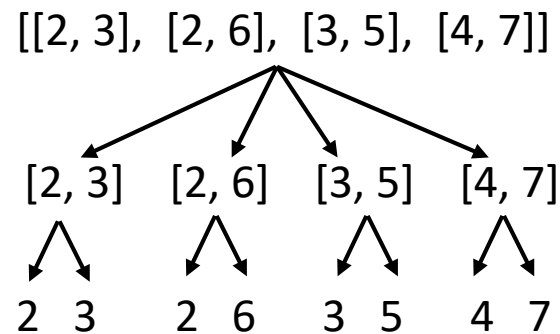
Reglas recursivas: Si $L_0, L_1, \dots, L_n \in \Sigma \cup LIST(\Sigma)$, entonces $[L_0, L_1, \dots, L_n]$ es una lista.

CONJUNTOS DEFINIDOS RECURSIVAMENTE

Árbol estructural y profundidad de una lista



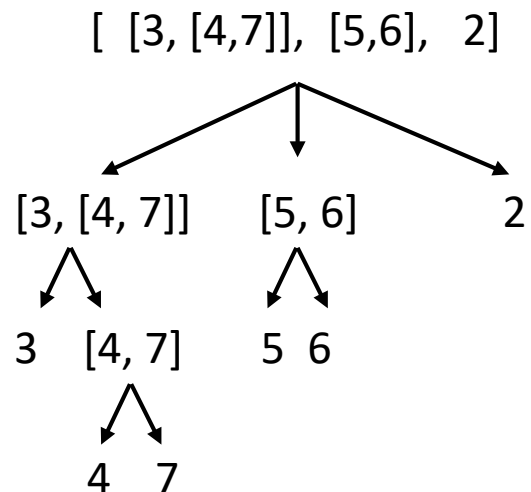
$P(L) = 1$



$P(L) = 2$

$[]$

$P(L) = 0$



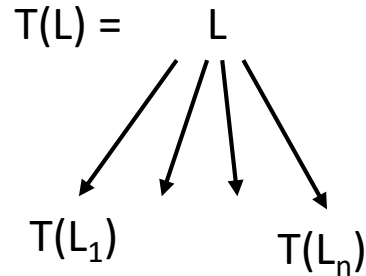
$P(L) = 3$

CONJUNTOS DEFINIDOS RECURSIVAMENTE

Se denomina **árbol estructural de** $L \in \Sigma \cup \text{LIST}(\Sigma)$ y se representa $T(L)$ al árbol definido (recursivamente) por:

$$T(L) = L$$

Si $L = []$ o bien $L \in \Sigma$



Si $L = [L_1, \dots, L_n]$

Se denomina **profundidad de una lista** a la **longitud** de la rama más larga de su árbol estructural.

La longitud de una rama de un árbol es el número de aristas de dicha rama.

FUNCIONES BÁSICAS SOBRE LISTAS

En la construcción de funciones recursivas sobre el conjunto de las listas, se usan algunas funciones básicas. Las más frecuentes son:

CAB: $\text{LIST}(\Sigma) - \{[]\} \rightarrow \Sigma \cup \text{LIST}(\Sigma)$
 $\text{CAB}(L)$ proporciona el primer elemento de L

RESTO: $\text{LIST}(\Sigma) - \{[]\} \rightarrow \text{LIST}(\Sigma)$
 $\text{RESTO}(L)$ proporciona la lista obtenida al borrar el primer elemento de L

CONC: $\text{LIST}(\Sigma) \times \text{LIST}(\Sigma) \rightarrow \text{LIST}(\Sigma)$
 $\text{CONC}(L1, L2)$ proporciona la lista obtenida al colocar los elementos de $L1$ en primer lugar y después los de $L2$. Notación: $\text{CONC}(L1, L2) = L1 \parallel L2$

LISTA: $\{ \text{palabras sobre } \{\Sigma, [,], , \} \} \rightarrow \{0, 1\}$
 $\text{LISTA}(L)$ devuelve 1 si L es lista y 0 en otro caso.

FUNCIONES BÁSICAS SOBRE LISTAS

Ejemplo: Dadas $L1=[2,4,6,8,0]$ $L2=[1,3,5,7]$ $L3=[[0,2],3,4,5]$

1. Obtener: $CAB(L1)$, $CAB(L3)$, $RESTO(L2)$, $RESTO(L3)$,
 $CAB(CAB(L3))$, $CONC(L1,L2)$, $CONC(L2,L3)$
2. Usar las funciones elementales adecuadas para obtener a partir de las listas anteriores:
 $[6,8,0]$ $[1,3,5,7,2,4,6,8,0]$ $[3,4,5]$ $[2]$

Para definir funciones recursivas sobre el conjunto de listas, también se suele utilizar la **función longitud**, que proporciona el número de elementos de una lista L .

$$LONG(L) = \begin{cases} 0 & \text{si } L = [] \\ 1 + LONG(RESTO(L)) & \text{e.o.c.} \end{cases}$$

FUNCIONES SOBRE LISTAS DEFINIDAS RECURSIVAMENTE

Ejemplo 1: Definir, de manera recursiva, una función que reciba un número natural n y construya una lista plana con los cuadrados de los números del 1 al n .

$$C: N \rightarrow LIST_P(N) \quad C(n) = \begin{cases} [0] & \text{si } n = 0 \\ C(n-1) || [n^2] & \text{si } n \geq 1 \end{cases}$$

Ejemplo 2: Definir, de manera recursiva, una función que reciba una lista plana de naturales y devuelva la suma de los elementos de la lista

$$S: LIST_P(N) \rightarrow N \quad S(L) = \begin{cases} 0 & \text{si } L = [] \\ CAB(L) + S(RESTO(L)) & \text{e. o. c.} \end{cases}$$

Ejemplo 3: Definir, de manera recursiva, una función que reciba una lista plana L de naturales y devuelva la lista que contiene los elementos de L en orden inverso.

$$I: LIST_P(N) \rightarrow LIST_P(N) \quad I(L) = \begin{cases} [] & \text{si } L = [] \\ I(RESTO(L)) || [CAB(L)] & \text{e. o. c.} \end{cases}$$

Ejercicio: hallar $C(3)$, $S([3, 4, 2])$ y $I([3, 4, 2])$ usando las definiciones anteriores

FUNCIONES SOBRE LISTAS DEFINIDAS RECURSIVAMENTE

Ejemplo 4: Definir, de manera recursiva, una función que reciba una lista de naturales y devuelva la suma de los elementos de la lista

Caso 1 : $S: LIST_P(N) \rightarrow N$

$$S(L) = \begin{cases} 0 & \text{si } L = [] \\ CAB(L) + S(RESTO(L)) & \text{e. o. c.} \end{cases}$$

Extensión: $S: LIST(N) \rightarrow N$

$$S(L) = \begin{cases} 0 & \text{si } L = [] \\ CAB(L) + S(RESTO(L)) & \text{si } L \neq [] \text{ y } LISTA(CAB(L)) = 0 \\ S(CAB(L)) + S(RESTO(L)) & \text{si } L \neq [] \text{ y } LISTA(CAB(L)) = 1 \end{cases}$$

Ejercicio: Hallar $S([3,4,2])$ y $S([2, [3,6], [3,4,2]])$ según las definiciones anteriores.